in28minutes.com · Interview Preparation Series

TERRAFORM

Interview Guide

5 Chapters · 37 Questions & Answers · June 2026

Table of Contents

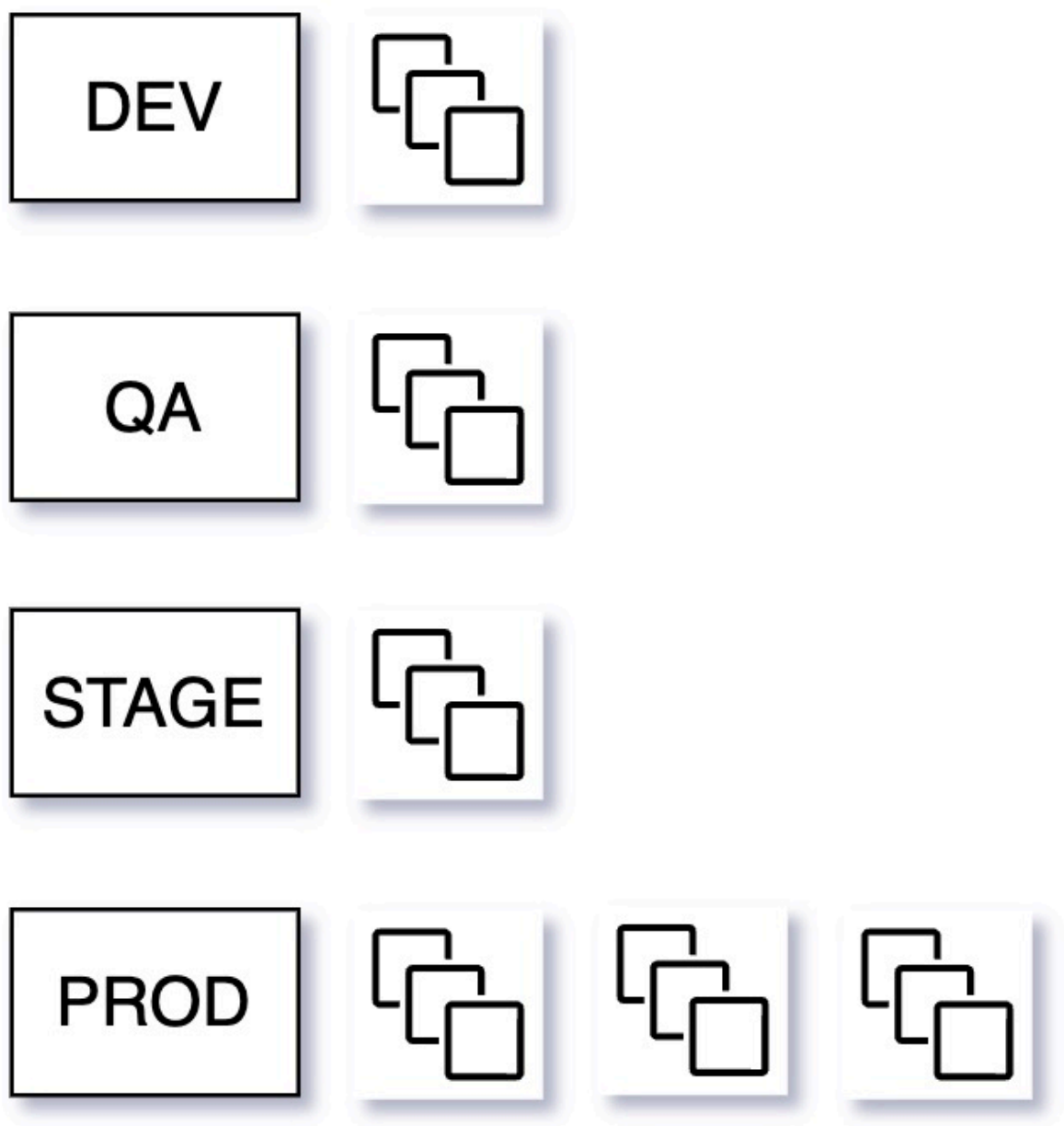
CHAPTER 1

Introduction to Terraform

9 Questions & Answers

1 Introduction to Terraform

Q1 What is Terraform?



Environments Are Not All The Same

- Understanding Need for Infrastructure Automation: Manually creating servers, networks, databases, and permissions for every environment (Dev, QA, Prod) is slow and error-prone
- Terraform: An open-source tool that lets you define and provision cloud infrastructure using simple, declarative configuration files
- Solves for Repeatability and Consistency: Infrastructure is defined as code and can be version-controlled, reviewed, and reused across teams and environments
- Eliminates Manual Infrastructure Setup: Automates the provisioning of servers, databases, and networks, reducing manual errors and delays
- Works Across Cloud Providers and Services: Supports AWS, Azure, Google Cloud, Kubernetes, GitHub, and more using the same tool and very similar syntax
- Enables Version Control for Infrastructure: Stores infrastructure as code in Git, allowing tracking of changes
- Improves Team Collaboration and Workflow: Infrastructure changes can be reviewed, tested, and approved just like application code
- Integrates Easily with CI/CD Pipelines: Enables automatic infrastructure updates as part of your DevOps workflows
- Reduces Cloud Costs Through Automation: Automatically tears down unused infrastructure without missing any resource!

Q2 When should you use and when should you NOT use Terraform?

- When should you NOT use Terraform?

When You Need Fine-Grained Configuration Management: Tools like Ansible, Chef, or Puppet are better suited for managing OS configurations, package installations, and runtime processes

When Your Infrastructure is Small and Rarely Changes: For very simple setups with minimal updates, Terraform may be overkill and add unnecessary complexity

When You Prefer GUI-Based Provisioning: If your team is non-technical or prefers clicking through AWS/Azure/Google Cloud consoles, Terraform's code-first approach may not be ideal

When You Want Quick One-Time Changes: For quick, temporary changes, using the cloud provider console or CLI may be faster than writing and applying Terraform code

When Your Team Lacks Git Practices: Terraform works best when used with version control workflows — without them, it might not work well

- When should you use Terraform?

When You Want to Automate Complex Cloud Infrastructure: Ideal for setting up VMs, databases, networks, and security groups automatically instead of doing it manually

When You Need Repeatable Environments: Useful for creating identical Dev, QA, and Prod environments to reduce environment-specific problems

When You're Managing Multi-Cloud or Hybrid Cloud: Works across AWS, Azure, Google Cloud, and on-premise — great for teams using more than one platform

When Your Team Wants to Use Infrastructure as Code (IaC): Allows storing infrastructure definitions in Git for better collaboration, review, and rollback

When You Want to Integrate Infrastructure into CI/CD Pipelines: Terraform fits well into automated pipelines for faster and safer deployments

When You're Adopting Immutable Infrastructure Practices: Makes it easier to recreate infrastructure from scratch instead of patching it manually

---

Q3 How does Terraform compare to other tools that help with Infrastructure as Code?

- AWS CloudFormation: Native to AWS; allows defining infrastructure using JSON/YAML. Best for deep AWS integrations but lacks multi-cloud flexibility
- Azure Resource Manager (ARM) Templates / Bicep: Native IaC tools for Azure; Bicep improves readability over ARM's verbose JSON. Best for Azure-specific projects.
- Google Cloud Deployment Manager: Google Cloud-native tool using YAML and Jinja2 templates. Suitable for Google Cloud-only setups but limited outside Google Cloud.
- Pulumi (Code-Driven IaC): Allows writing infrastructure using familiar languages like Python, JavaScript, TypeScript, Go. Best for developers who want to automate IaC using a familiar programming language. (No need to learn HCL!)
- Ansible: Primarily for configuration management (configuring and managing applications, processes and files on existing servers) BUT can also create infrastructure (using cloud modules - not recommended!)
- Chef / Puppet: Focused on configuring and managing servers after they're provisioned. Less commonly used for full infrastructure provisioning.

Q4 Compare Terraform vs Ansible

Feature	Terraform
Purpose	Infrastructure provisioning and management
Type	Declarative – Describe desired state. Terraform would compare and apply changes.
Primary Use	Creating, modifying, and deleting cloud resources
Mostly Used Language/Format	HashiCorp Configuration Language (HCL)
State Management	Maintains a state file to track infrastructure changes
Provisioning Speed	Efficient for creating and managing infrastructure at scale
Infrastructure Focus	Best suited for provisioning infra on cloud providers like AWS, Azure, Google Cloud
Configuration Management	Limited configuration management capabilities
My Recommendations	Use Terraform for Provisioning

Q5 How can you use Terraform to create resources?

- (Step 1) Write HCL Configuration Files: Use .tf files to define infrastructure using HashiCorp Configuration Language
- (Step 2) Initialize the Terraform Project: Run terraform init to download necessary provider plugins and set up the local working directory
- (Step 3) Validate the Configuration: Use terraform validate to check for syntax errors or misconfigurations before any action is taken
- (Step 4) Preview Infrastructure Changes: Run terraform plan to see what Terraform intends to create, update, or destroy — helps prevent surprises
- (Step 5) Apply the Configuration to Create Resources: Run terraform apply to actually provision the defined infrastructure after reviewing the plan
- (Step 6) Make Changes and Reapply: Modify .tf files when requirements change, and repeat the plan apply steps to update infrastructure
- (Step 7) Optional Cleanup with Destroy: Use terraform destroy to tear down all infrastructure
- Declarative and Predictable: You define the desired outcome — Terraform figures out the how to make it actual!

HCL / Terraform

```
# AWS Provider Configuration
# WHAT: Defines which cloud provider to use.
# WHY: Needed for Terraform to know how to create
#      resources

provider "aws" {
  region = "us-east-1" # Choose AWS region
}

# S3 Bucket Resource Configuration
# WHAT: Creates an AWS S3 bucket
# WHY: To store data/objects in AWS (like backups,
#      static files, logs etc.)

resource "aws_s3_bucket" "my_s3_bucket" {
  bucket = "my-s3-bucket-in28minutes-rangake-002"
  # Bucket name must be globally unique

  versioning {
    enabled = true
    # Enables object versioning to keep history
  }

  # tags = {
  #  project = "my-first-terraform-project"
  # }
}

# IAM User Resource Configuration
# WHAT: Creates a new IAM user in AWS

resource "aws_iam_user" "my_iam_user" {
  name = "my-iam-user"
}
```

Q6 Why is Terraform called a Declarative Tool?

- How Terraform Works: Terraform uses a declarative approach to define infrastructure as code, compare it with the real-world state, and apply only the necessary changes to match the desired outcome
- Avoids Step-by-Step Instructions: You describe what you want, not how to get there — Terraform figures out the steps
- (Declarative) Define Desired State Using HCL: You write .tf files to describe the end result (e.g., an EC2 instance should exist with a certain type and AMI)
- (Plan) Terraform Creates an Execution Plan: It compares your configuration with the current infrastructure state and shows what it will create, update, or delete
- (Apply) Executes Only the Required Changes: Terraform applies the minimal set of actions needed to bring real infrastructure in sync with your code
- (State) Keeps Track of Resources in a State File: Maintains a snapshot of managed resources for future comparisons and updates
- (Key Difference: Imperative vs Declarative):

  Imperative (e.g., Bash, Ansible): You tell the system exactly how to do each step (e.g., create subnet then launch VM)

  Declarative (Terraform): You declare the final outcome, and the tool decides the best way to achieve it
- (Advantage) Safer and More Predictable Changes: Declarative logic avoids manual sequencing and reduces errors

  Execution Plan Visibility: Terraform provides a detailed preview of all actions (create, update, delete) before they're applied - This lets you catch unintended consequences (like deleting a production resource or altering a critical security group) before execution

  Idempotent Behavior: Running the same configuration multiple times yields the same result, with no additional changes — Terraform won't recreate or modify resources that are already aligned with the desired state

  Minimizes Human Error Through Automation: Since Terraform computes resource dependencies and action ordering automatically, there's no need to write imperative step-by-step logic — reducing the risk of mistakes during provisioning or teardown

  Makes Creating New Environments Safer: All changes are driven by the same version-controlled codebase. This ensures that every environment (dev, stage, prod) is built with the same structure and settings — making deployments consistent and repeatable

Q7 Explain a Few Commands That Are Frequently Used with Terraform

- terraform validate – Check for Syntax Errors Early: Verifies that the .tf files are syntactically correct
- terraform fmt – Enforce Consistent Formatting: Automatically formats your Terraform files for readability and consistency
- terraform init – Prepare the Project Directory: Initializes the working directory, downloads required provider plugins, and sets up the backend for storing terraform state if defined
- terraform plan – Preview Infrastructure Changes: Shows what Terraform will do — what resources will be added, changed, or destroyed — before you apply it
- terraform apply – Create or Update Resources: Executes the changes shown in the plan, provisioning or modifying cloud infrastructure as described in your code
- terraform destroy – Tear Down Infrastructure: Removes all resources defined in the configuration — useful for cleaning up test environments or demos

Q8 What is the Terraform Plugin Architecture?

- Understanding Need for Extensibility and Multi-Cloud Support: Infrastructure teams need to provision resources across AWS, Azure, Google Cloud, Kubernetes, and SaaS providers — Terraform must support all of them without bloating up the core tool
- What is Terraform Plugin Architecture: A modular system where Terraform's core communicates with separate provider plugins to manage external infrastructure
- Core Binary is Lightweight and Generic: The terraform CLI focuses on the core logic — it delegates all resource-specific tasks to plugins
- Enables Flexible Integration with Different Platforms: Each provider knows how to talk to a specific platform (like AWS or GitHub)
- Providers Are Managed Externally: Each provider (e.g., hashicorp/aws, azure, google, kubernetes) is a separate binary downloaded during terraform init
- Build Your Own Custom Providers: Teams can build custom provider for internal platforms
- Keeps Terraform Core Clean and Scalable: New cloud services can be supported by updating or adding plugins — no need to change Terraform's core

```
HCL / Terraform
```

```
#  AWS Provider
# Used to create AWS infrastructure like EC2, S3
provider "aws" {
  region = "us-east-1"
}

#  AzureRM Provider
# Used to manage Azure resources (VMs, Storage, etc)
provider "azurerm" {
  features {}
  # 'features' block is mandatory even if empty
}

#  Google Cloud Provider
# Manages Google Cloud resources like GCE, GCS, Cloud Run
provider "google" {
  project = "my-Google Cloud-project"
  region  = "us-central1"
}

#  Docker Provider
# WHAT: Used to manage Docker containers/images
provider "docker" {
  # Connects to local Docker daemon
}

#  GitHub Provider
# WHAT: Manage repos, teams, ect on GitHub
provider "github" {
  token = var.github_token
  owner = "in28minutes"
}

#  Kubernetes Provider
# WHAT: Manage K8s resources (pods, services, etc.)
# HOW: Connect using kubeconfig file
provider "kubernetes" {
  config_path = "~/.kube/config"
}

#  Notes
# - Add `required_providers` in `terraform {}` block
#   to pin versions (recommended)
```

Q9 How Can Terraform Providers Authenticate to Cloud Platforms?

Authentication Methods: Terraform providers must authenticate securely to cloud platforms to manage infrastructure

- Recommended methods: Using Environment Variables (ideal for CI/CD), Shared Credential Files (convenient for local development), or IAM Roles (best practice for automated environments in clouds)
- Hard-coding in Code is NOT Recommended: Hard-coding credentials directly in the provider block is strongly discouraged

```
HCL / Terraform
```

```
#  METHOD 1: Environment Variables
# WHAT: Use OS-level variables to pass secrets
# WHEN: Recommended for CI/CD or local dev
# WHERE: Set in terminal or CI pipeline env config

# AWS EXAMPLE
# export AWS_ACCESS_KEY_ID="your-key"
# export AWS_SECRET_ACCESS_KEY="your-secret"
# export AWS_REGION="us-west-2"

provider "aws" {
  # No credentials needed here if ENV vars are set
}

#  METHOD 2: Shared Credential Files
# WHAT: Use provider CLI config files (e.g., AWS)
# WHY: Easy to use for local development
# Use "aws configure" to set the file up
provider "aws" {
  region                   = "us-west-2"
  shared_credentials_files = ["~/.aws/credentials"]
  profile                  = "default"
}


#  METHOD 3: IAM Roles / Service Principals
# WHAT: Assume identity from the running instance
# WHY: Best practice for running automation in cloud
# WHEN: Use inside EC2 or Cloud Shell in cloud

# AWS IAM ROLE ASSUMPTION
provider "aws" {
  region = "us-west-2"
  assume_role {
    role_arn     = "arn:aws:iam::123456789012:role/TerraformRole"
    session_name = "terraform-session"
  }
}


#  METHOD 4: Hardcoded in Provider Block (NOT RECOMMENDED)
# WHAT: Explicitly specify access credentials
# WHEN: Only for learning or quick testing

# AWS EXAMPLE ( Avoid in real projects)
provider "aws" {
  region     = "us-west-2"
  access_key = "AKIAIOSFODNN7EXAMPLE"
  secret_key = "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY"
}

#  RECOMMENDED BEST PRACTICE
# - NEVER commit access keys in code
```

CHAPTER 2

Terraform Language - HCL

8 Questions & Answers

2 Terraform Language - HCL

Q1 Why did HashiCorp create the HCL (HashiCorp Configuration Language)?

- Understanding Need for a Purpose-Built Language: General-purpose languages/formats (like YAML, JSON, or Python) are not optimized for describing infrastructure
- HCL (HashiCorp Configuration Language): HCL was designed specifically to be human-readable, machine-friendly, and domain-specific for Infrastructure as Code (IaC)
- Easy to Learn, Hard to Misuse: Simple for beginners, with guardrails to prevent critical errors
- Improve Readability for Operators and DevOps Teams: HCL uses a clean, block-based syntax that's easier to read and understand - even for folks unfamiliar with programming
- Provide Strong Static Validation: Terraform can detect syntax and type errors before applying any changes — reducing costly mistakes
- Enable Rich, Custom Types and Expressions: HCL supports built-in functions, conditionals, for-loops, maps, lists, and interpolation

Q2 What are the core concepts of HCL? Explain with a practical example.

- Providers: Allows Terraform to connect to and manage resources on a specific cloud or service (e.g., AWS, Azure, Google Cloud, Docker, Kubernetes)
- Resources: Allows Terraform to create, update, or delete actual infrastructure components (e.g., IAM user, S3 bucket)
- Variables: Allows users to input values from the outside, making configurations flexible and reusable
- Locals: Allows defining reusable internal constants or computed values (CANNOT be overridden from outside)
- Output: Allows Terraform to display and share useful result values

```
HCL / Terraform
```

```
#  VARIABLE BLOCK
# WHAT: Accepts IAM user name input
# WHY: Makes the config reusable

variable "iam_user_name" {
  type    = string
  default = "default_iam_user_name_value"
  # Can be overridden using terraform.tfvars or CLI
  # Example:
  # terraform apply -var="iam_user_name=something_else"
}

#  LOCALS BLOCK
# WHAT: Used for internal computed values
# WHY: Avoid repetition

locals {
  organization = "in28minutes" # Local value. Will NOT be changed from outside
}

#  PROVIDER BLOCK
# WHAT: Configures AWS access
# WHY: Required to manage AWS resources

provider "aws" {
  region = "us-east-1"
}

#  RESOURCE BLOCK
# WHAT: Creates a single IAM user
# WHY: Provision identity in AWS

resource "aws_iam_user" "my_user" {
  name = var.iam_user_name + "_" + local.organization
  # Interpolation syntax NOT necessary for simple things
  # ${var.iam_user_name} + "_" + ${local.organization}
}

#  OUTPUT BLOCK
# WHAT: Displays the IAM user name
# WHY: Helps confirm what was created
#      Share output with other projects

output "iam_user_name_value" {
  value       = aws_iam_user.my_user.name
  description = "The IAM user that was created"
}
```

Q3 Why do we need variables in Terraform?

- Variable Declaration: Variables are declared using variable blocks with optional type constraints, default values, descriptions, and validation rules
- Variable Reference: Variables are referenced using var.<variable_name> syntax throughout the configuration
- Primitive Data Types: Supports string, number, and bool as fundamental data types
- Complex Data Types: Supports list, set, map, object, and tuple for structured data
- Type Validation: Type constraints help catch errors early and provide better documentation for expected input formats

Variable assignment methods:

- Command line: terraform apply -var="region=us-east-1"
- Environment variable: export TF_VAR_region="us-east-1" (TF_VAR_ + VariableName)
- Values defined in terraform.tfvars file
- Custom Variable file: terraform apply -var-file="prod.tfvars"

```
HCL / Terraform
```

```
#  VARIABLE DECLARATION
# WHAT: Declares a variable named "region"
# WHY: Makes config reusable with flexible input

variable "region" {
  type        = string
  default     = "us-east-1"
  description = "AWS region to deploy into"

  # Variation: Add validation rules (optional)
  validation {
    condition     = contains(
                    ["us-east-1", "us-west-1"], var.region)
    error_message = "Allowed values: us-east-1, us-west-1"
  }
}

#  VARIABLE REFERENCE
# WHAT: Refer to variables using var.<name>

provider "aws" {
  region = var.region
}

#  PRIMITIVE DATA TYPES
# string, number, bool

variable "app_name" {
  type        = string
  default     = "my-app"
}

variable "instance_count" {
  type        = number
  default     = 2
}

variable "enable_monitoring" {
  type        = bool
  default     = true
}

#  EXAMPLES OF COMPLEX DATA TYPES

# list of strings
variable "azs" {
  type    = list(string)
  default = ["us-east-1a", "us-east-1b"]
}

# map of string values
variable "tags" {
  type = map(string)
  default = {
    owner = "team-a"
    env   = "dev"
  }
}

# set of strings
variable "unique_zones" {
  type    = set(string)
  default = ["us-east-1a", "us-east-1b"]
}

#  TYPE VALIDATION BENEFIT
# Ensures incorrect input types fail early
# terraform validate will catch mismatches

#  VARIABLE ASSIGNMENT METHODS (IN PRIORITY ORDER)

# 1 Command Line
```

```
# terraform apply -var="region=us-east-1"
# terraform apply -var-file="prod.tfvars"

# 2 Environment Variable
# export TF_VAR_region="us-east-1"

# 3 terraform.tfvars
# region = "us-east-1"

# 4 Default in variable block
# variable "region" {
#   default = "us-west-1"
# }
```

Q4 What is the difference between locals and variables in Terraform?

- Locals Block: A locals block lets you define one or more named local values (locals) within a module
- Purpose and Benefits

  Avoid Repetition: Define complex expressions or values once instead of repeating them across resources in the same module

  Enhance Readability: Give meaningful names to complex expressions making code self-documenting
- (Key Difference) Variables Are Directly Configurable From Outside:

  Variables = external input (flexible)

  Locals = internal values or constants

Code

```
################################################
#  LOCALS BLOCK
# WHAT: Internal reusable values
# WHY: Avoid repetition, simplify logic
# WHEN: Use for naming, tagging, timestamps
################################################

locals {
  # App name used across resources
  app_name = "inventory"

  # IAM role name: app + env + suffix
  lambda_role_id = "${local.app_name}-\
${var.environment}-lambda-role"

  # Timestamp in YYYYMMDDhhmmss format
  timestamp = formatdate(
    "YYYYMMDDhhmmss",
    timestamp()
  )

  # List of standard ports (SSH, HTTP, HTTPS)
  default_ports = [22, 80, 443]

  # Shared tagging across all AWS resources
  common_tags = {
    Owner       = "ranga"
    Environment = var.environment
    Project     = var.project
  }
}
```

Q5 What is a Data Source in Terraform?

- Need For Data Source: Sometimes we need to reference resources that were not created by Terraform — like an existing VPC, subnet, or AMI
- What is a Data Source in Terraform: A read-only block that allows you to fetch and use information about existing resources in your configuration
- Helps You Integrate with Existing Infrastructure: Enables re-usability, avoids duplication
- Common Use Cases:

  Get latest AMI ID from AWS

  Look up existing VPCs, subnets, or security groups

  Share information between separate Terraform projects (You want to get information about a resource created in a different project)
- Read-Only Access to External Resources: Data sources never modify infrastructure — they only retrieve data
- Reduces Hard-coding: Avoids hard-coding of resource details

```
HCL / Terraform
```

```
#  DATA SOURCE SYNTAX:
# data "<provider>_<type>" "<name>" { ... }

#  WHEN: Use when you need to reference existing
# resources not created in your Terraform code.

#  DATA SOURCE EXAMPLE: AWS AMI Lookup
# WHAT: Find the latest Amazon Linux AMI
# WHY: Use it for EC2 instance launch

data "aws_ami" "amazon_linux" {
  most_recent = true

  owners = ["137112412989"] # Official Amazon Linux AMI owner

  filter {
    name   = "name"                  # Which AMI attribute to filter on
    values = ["al2023-ami-*-x86_64"] # Defining a Wild Card Search
  }

  # ADD OTHER FILTERS AS NEEDED

}

#  The data block does NOT create the AMI
# It just fetches its ID and metadata

#  RESOURCE USING DATA SOURCE
# Launch an EC2 instance using that AMI

resource "aws_instance" "example" {
  ami           = data.aws_ami.amazon_linux.id
  instance_type = "t3.micro"

  tags = {
    Name = "data-source-example"
  }
}

#  OUTPUT: Print the AMI ID fetched via data source

output "selected_ami_id" {
  value       = data.aws_ami.amazon_linux.id
  description = "AMI ID fetched using data source"
}

#  VARIATION: Lookup existing VPC
# data "aws_vpc" "default" {
#   default = true
# }

#  VARIATION: Lookup S3 bucket (already created)
# data "aws_s3_bucket" "mybucket" {
#   bucket = "my-existing-bucket-name"
# }
```

Q6 How do resources compare to data sources in Terraform?

- resource – Actively Manages Infrastructure:
  Creates, updates, or destroys infrastructure
  Terraform tracks and owns the complete lifecycle
  Example:
- data – Reads Existing Information:
  Fetches details about resources not created by Terraform
  Does not change or manage the resource
  Read ONLY!
  Example:
- (Best Practice) Use data blocks to Dynamically Fetch IDs: Avoid hardcoding values like AMI IDs, subnet IDs, or security group names — use data instead for flexibility and maintainability

Aspect	Resource Block
Keyword	resource
Primary Action	create, update, delete real infrastructure
Lifecycle Phases	Planned in plan, executed in apply
Typical Use Cases	Provision VMs, networks, DNS records, managed services

HCL / Terraform

```
resource "aws_s3_bucket" "my_bucket" {
  bucket = "my-app-bucket"
}
```

HCL / Terraform

```
resource "aws_s3_bucket" "my_bucket" {
  bucket = "my-app-bucket"
}
```

Code

```
data "aws_s3_bucket" "existing_bucket" {
  bucket = "shared-logs"
}
```

Code

```
data "aws_s3_bucket" "existing_bucket" {
  bucket = "shared-logs"
}
```

Q7 What is the purpose of Provisioners in Terraform?

- Understanding Need for Custom Configuration After Provisioning: Sometimes, we need to install packages, run scripts, or perform setup steps after a VM or resource is created
- Provisioners: Provisioners allow you to run scripts or commands on a resource after it's created or before it's destroyed
- Used to Perform Setup Tasks Not Supported Natively by Terraform: Like bootstrapping applications, setting file permissions, or executing install commands
- Simple for Quick Use Cases: Easy to install packages or echo output without external tooling
- (Feature) Supports local-exec and remote-exec:
  - local-exec runs commands on your local machine
  - remote-exec runs commands inside the created VM using SSH or ..
- Helpful When Configuration Tools Aren’t Used: Good for lightweight setup tasks when Ansible, Chef, or Puppet are not being used
- (Best Practice) Use Sparingly and Prefer External Tools: Use Ansible or similar options when possible; reserve provisioners for unavoidable tasks only

```
HCL / Terraform
```

```
resource "aws_instance" "web_server" {
  ami           = var.latest_ami
  instance_type = "t3.micro"
  key_name      = "default-ec2"
  vpc_security_group_ids = [aws_security_group
                            .web_sg.id]

  #  HOW: Terraform connects to instance via SSH
  connection {
    type        = "ssh"
    host        = self.public_ip
    user        = "ec2-user"
    private_key = file(var.aws_key_pair)
  }

  #  PROVISIONER BLOCK - remote-exec
  # WHAT: Run commands inside the EC2
  # WHY: Automate post-launch setup
  # WHEN: Runs only during `terraform apply`
  # WHERE: Runs from Terraform machine to EC2
  provisioner "remote-exec" {
    inline = [
      # Install Apache
      "sudo yum install httpd -y",
      # Start Apache service
      "sudo service httpd start",
      # Create basic web page
      "echo Welcome to in28minutes - Virtual \
Server is at ${self.public_dns} | sudo tee \
/var/www/html/index.html"
    ]
  }

}



#  LOCAL-EXEC ON BUCKET CREATION
# WHAT: Runs echo locally when S3 is created
# HOW: Uses a null_resource with a trigger
# WHEN: After new bucket creation

resource "aws_s3_bucket" "my_bucket" {
  bucket = "in28minutes-demo-bucket-1234"
  # Replace with unique name (globally unique)
}

resource "null_resource" "after_s3_created" {
  provisioner "local-exec" {
    command = "echo Hello from Terraform!"
  }

  #  Trigger rerun when bucket id changes
  triggers = {
    bucket_id = aws_s3_bucket.my_bucket.id
  }
}
```

Q8 How do you create an EC2 instance using Terraform?

- Provision an EC2 instance: In the default VPC with a security group
- Install and start Apache Web Server: Host a simple welcome page

```
HCL / Terraform
```

```
provider "aws" {
  region = "us-east-1"
}

#  DATA SOURCES
# WHAT: Fetch existing resources (read-only)

#  Gets the default VPC (used in SG & subnet)
resource "aws_default_vpc" "default" {}

#  Get all default subnets from the default VPC
data "aws_subnets" "default_subnets" {
  filter {
    name   = "vpc-id"
    values = [aws_default_vpc.default.id]
  }
}

#  Get the latest Amazon Linux AMI
data "aws_ami" "amazon_linux" {
  most_recent = true

  owners = ["137112412989"] # Official Amazon Linux AMI owner

  filter {
    name   = "name"
    values = ["al2023-ami-*-x86_64"]
    #  Use wildcard for versioning like:
    # "al2023-ami-2023.*-x86_64"
  }

  # ADD OTHER FILTERS AS NEEDED

}

#  SECURITY GROUP
# WHAT: Allow HTTP and SSH into the instance
# WHY: Without this, the server is unreachable
resource "aws_security_group" "http_server_sg" {
  name   = "http_server_sg"
  vpc_id = aws_default_vpc.default.id

  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
    # Allows HTTP from anywhere
    # BEST PRACTICE: RESTRICT IPS
  }

  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
    # Allows SSH access (Use carefully)
    # BEST PRACTICE: RESTRICT IPS
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = -1
    cidr_blocks = ["0.0.0.0/0"]
    # Allows all outbound traffic
  }

  tags = {
    name = "http_server_sg"
  }
}
```

```
################################################
#  EC2 INSTANCE
# WHAT: Launches a virtual server in AWS
# WHY: Host a sample web page using Apache
################################################
resource "aws_instance" "http_server" {
  ami                    = data.aws_ami
                           .amazon_linux.id
  key_name               = "default-ec2"
  instance_type          = "t3.micro"
  vpc_security_group_ids = [
    aws_security_group.http_server_sg.id
  ]
  subnet_id = data.aws_subnets
              .default_subnets.ids[0]
  #  Uses the first subnet from default VPC

  #  SSH connection setup
  connection {
    type        = "ssh"
    host        = self.public_ip
    user        = "ec2-user"
    private_key = file(var.aws_key_pair)
  }

  #  Provisioner installs & starts Apache
  provisioner "remote-exec" {
    inline = [
      "sudo yum install httpd -y",
      "sudo service httpd start",
      "echo Welcome to in28minutes - Virtual\
       Server is at ${self.public_dns} | \
       sudo tee /var/www/html/index.html"
    ]
  }

}

variable "aws_key_pair" {
  # Provide value while executing
  # `terraform apply -var="aws_key_pair=PATH_TO_FILE\NAME.pem"`
}

output "http_server_public_dns" {
  value = aws_instance.http_server.public_dns
  #  Use this DNS to access the webpage
}
```

CHAPTER 3

Terraform State Management

8 Questions & Answers

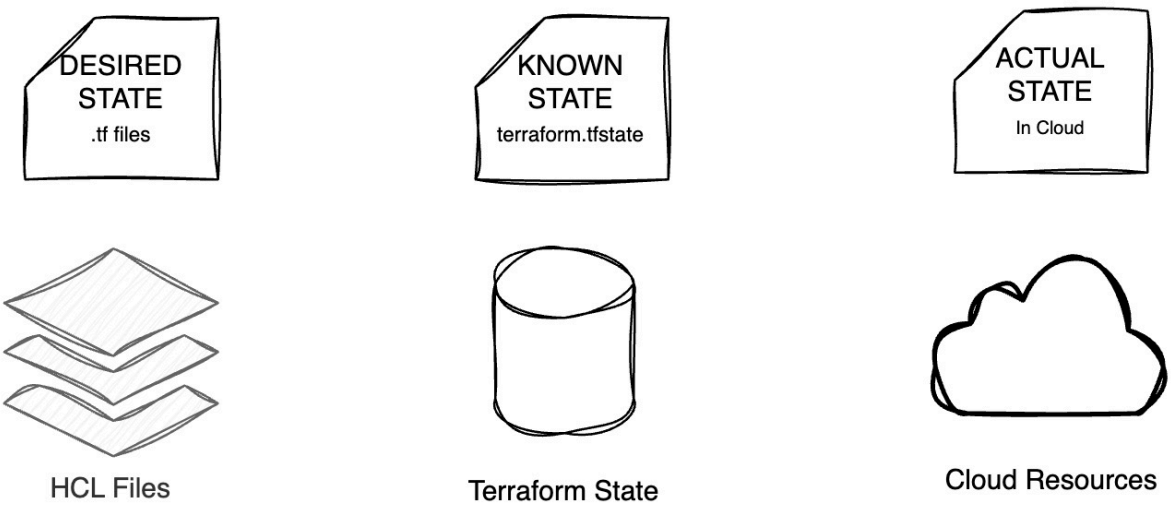
3 Terraform State Management

Q1 What is the need for Terraform State?

- Understanding Need for Tracking Infrastructure Changes: Without tracking, Terraform wouldn't know what resources were created earlier - If you run terraform apply, it might recreate resources again!
- What is Terraform State: A snapshot of the real-world infrastructure Terraform manages, stored in a file (terraform.tfstate)
  - Stores Metadata and Resource Attributes: Includes resource details - IDs, names, IPs, and other details
- Needed to Safely Plan, Update, and Destroy Infrastructure: Helps Terraform make accurate decisions during plan and apply operations
- Enables Accurate Change Detection: Compares your .tf configuration with the current state to show only what's changed
- Critical for Drift Detection: Helps Terraform identify when real-world infrastructure has been changed outside Terraform (Drift - A resource created by terraform is changed manually directly in the cloud platform)
- Key Terraform Commands: That Might Modify Terraform State
  - terraform apply: Creates, updates, or deletes infrastructure and updates the state file
  - terraform destroy: Deletes all managed resources and removes them from the state file
  - terraform refresh: Fetches real-world resource values and updates the state file without changing infrastructure

Q2 What are Desired, Known and Actual States?

Terraform State



- Desired State – What You Define in Code:
  - Defined in .tf files
  - Represents what you want your infrastructure to look like
  - Example: “I want 2 EC2 instances with type t3.micro in us-east-1”
- Known State – What Terraform Remembers: [CALLED "Terraform State"]
  - Stored in terraform.tfstate
  - Represents what Terraform thinks is currently deployed in the cloud platform
  - Includes resource IDs, IPs, tags, metadata, etc.
  - Example: “Terraform created 2 EC2 instances last time and here are their IDs and IPs”
- Actual State – What Really Exists in the Cloud:
  - The real, live infrastructure in your cloud account
  - Could differ if someone made changes outside Terraform (manual edits or scripts)
  - This is called Drift (Drift is NOT good)

Scenario 1: No State Exists – New Resource

- (Step 1) Known State in terraform.tfstate:
  - No .tfstate file exists yet, so Terraform has no record of any resources
  - This means the Known State is empty
- (Step 2) Define Desired State in Code:
  - You define a new resource in code, such as an S3 bucket
  - Example: resource "aws_s3_bucket" "demo" { bucket = "my-first-bucket" }
  - This becomes the Desired State
- (Step 3) Plan: You run terraform plan
- (Step 3A) Compare Desired vs Known State:
  - Terraform notices the S3 bucket is not present in the known state
  - It marks the bucket for creation
- (Step 3B) Query the Actual State from Cloud Provider:
  - Terraform checks with AWS to confirm whether the bucket already exists
  - This is the Actual State, and it's empty too (no such bucket exists)
- (Step 3C) Generate Execution Plan:
  - Terraform compares: Desired Known AND Actual is empty
  - It prepares a plan to create the S3 bucket
- (Step 4) Apply: You run terraform apply
- (Step 4A) Bucket Created:
  - The bucket is created in AWS (Actual State Updated to Desired State)
- (Step 4B) Known State Updated:
  - The .tfstate file is updated to record this new bucket in the Known State

Scenario 2: Resource Updated in Code – Enabling Versioning

- (Step 1) Known State in terraform.tfstate:
  The .tfstate file shows that an S3 bucket named "my-first-bucket" exists with no versioning block
  This represents the current Known State
- (Step 2) Update Desired State in Code:
  You update the .tf file to enable versioning on the S3 bucket:
  This becomes the new Desired State
- (Step 3) Plan: You run terraform plan
- (Step 3A) Compare Desired vs Known State:
  Terraform detects that versioning is now required (in Desired) but not present (in Known)
  It marks the bucket for update
- (Step 3B) Query the Actual State from Cloud Provider:
  Terraform queries AWS to get the live state of the bucket
  It confirms that versioning is not enabled
- (Step 3C) Generate Execution Plan:
  Terraform compares: Desired Known AND Known = Actual
  It prepares an update to enable versioning on the S3 bucket
- (Step 4) Apply: You run terraform apply
- (Step 4A) Bucket Updated:
  Terraform enables versioning on the S3 bucket in AWS
  The Actual State is now aligned with the Desired State
- (Step 4B) Known State Updated:
  The .tfstate file is updated to reflect that versioning is enabled
  The new Known State now matches the Desired and Actual states

Scenario 3: Manual Change in Console – Drift Detected

- (Step 1) Known State in terraform.tfstate:
  The .tfstate file shows that an S3 bucket named "my-first-bucket" exists with versioning = true
  This is the Known State
- (Step 2) Desired State in Code (Unchanged):
  Your .tf file still expects versioning to be enabled:
  This is the Desired State (same as last apply)
- (Step 3) Manual Change in Console:
  Someone disables versioning in the AWS Management Console
  Now the Actual State in the cloud is versioning = false
- (Step 4) Plan: You run terraform plan
- (Step 4A) Compare Desired vs Known State:
  Terraform sees no change in code, so Desired = Known
- (Step 4B) Query the Actual State from Cloud Provider:
  Terraform refreshes the actual state by querying AWS
  It detects that versioning is not enabled anymore
  This is the Actual State, and it does not match Known
- (Step 4C) Detect Drift Between Known and Actual:
  Since the Known State has versioning = true but the Actual State is false
  Terraform detects drift
- (Step 4D) Generate Execution Plan:
  Desired = Known Actual
  Terraform prepares a plan to re-enable versioning to fix the drift
- (Step 5) Apply: You run terraform apply
- (Step 5A) Bucket Reconfigured:
  Terraform enables versioning again on the bucket in AWS
  The Actual State is corrected to match Desired and Known
- (Step 5B) Known State Reconfirmed:
  .tfstate remains the same since versioning was already recorded as enabled
  Drift is now resolved

HCL / Terraform

```
resource "aws_s3_bucket" "demo" {
  bucket = "my-first-bucket"

  versioning {
    enabled = true
  }
}
```

HCL / Terraform

```
resource "aws_s3_bucket" "demo" {
  bucket = "my-first-bucket"

  versioning {
    enabled = true
  }
}
```

HCL / Terraform


```
resource "aws_s3_bucket" "demo" {
  bucket = "my-first-bucket"

  versioning {
    enabled = true
  }
}
```

HCL / Terraform

```
resource "aws_s3_bucket" "demo" {
  bucket = "my-first-bucket"

  versioning {
    enabled = true
  }
}
```


Q3 What are the Options for Storing Terraform State?

- Local Storage – ONLY for Beginners and Single Users:
  - State file is stored on your local machine in the working directory
  - Simple to set up, ideal for testing or solo use
  - Not safe for teams — can lead to drift and conflicts (Developer A runs terraform apply on their laptop and creates an EC2 instance, but Developer B doesn't have the updated .tfstate file, so their next plan shows the EC2 instance as missing and may try to recreate it)
- Remote Backend for Terraform State:
  - Amazon S3 Backend – Secure and Scalable for AWS Users: Stores state in an S3 bucket
  - Azure Blob Storage: Ideal for Azure-Based Projects
  - Google Cloud Storage: Best for Google Cloud Projects
  - HCP Terraform(Terraform Cloud): Hosted State Management by HashiCorp
  - Enables Team Collaboration: All team members and CI/CD tools work off the same central state
  - Adds Backup and Audit Capabilities: Remote backends often support history and recovery options

HCL / Terraform

```
# AWS S3 Remote Backend
terraform {
  backend "s3" {
    bucket         = "my-terraform-state-bucket"
    key            = "path/to/my/terraform.tfstate"
    region         = "us-west-2"
    dynamodb_table = "terraform-state-lock"
    encrypt        = true
  }
}

# -------------------------------------
# Azure Storage Backend
# terraform {
#   backend "azurerm" {
#     resource_group_name  = "terraform-state-rg"
#     storage_account_name = "terraformstatestorage"
#     container_name       = "tfstate"
#     key                  = "prod.terraform.tfstate"
#   }
# }

# -------------------------------------
# Google Cloud Storage (GCS) Backend
# terraform {
#   backend "gcs" {
#     bucket = "my-terraform-state-bucket"
#     prefix = "terraform/state"
#   }
# }

# -------------------------------------
# HCP Terraform Backend
# terraform {
#   cloud {
#     organization = "my-organization"
#     workspaces {
#       name = "my-workspace"
#     }
#   }
# }

# -------------------------------------
# Consul Remote Backend
# terraform {
#   backend "consul" {
#     address = "consul.example.com:8500"
#     scheme  = "https"
#     path    = "terraform/myproject"
#   }
# }
```

Q4 Practical Example: Remote Backend State Management + Locking

- S3 Bucket for State Backend: Create an S3 bucket to store the Terraform state file remotely, enabling team collaboration and state persistence
- Versioning for State Bucket: Enable object versioning on the S3 bucket to support rollback and history of state file changes
- Encryption for State Bucket: Apply server-side encryption (AES256) to protect sensitive state data at rest
- DynamoDB for State Locking: Create a DynamoDB table to manage state locks and prevent simultaneous modifications during Terraform operations

```
HCL / Terraform
```

```
# ------------------------------------------
#  PROVIDER BLOCK
# WHAT: Configures AWS as the target platform
# WHY: Needed to provision S3, DynamoDB, IAM

provider "aws" {
  region = "us-east-1"
}

# ------------------------------------------
#  STATE BACKEND BUCKET (S3)
# WHAT: Stores Terraform state file
# WHY: Enables collaboration, versioning, recovery

resource "aws_s3_bucket" "backend_state" {
  bucket = "my-app-backend-state"

  lifecycle {
    prevent_destroy = true
    # Prevents Terraform from destroying the resource
    # when terraform destroy is run
    # Prevent accidental deletion of state bucket
  }
}

# ------------------------------------------
#  VERSIONING FOR STATE BUCKET
# WHAT: Enables history of state changes
# WHY: Helps rollback or debug state
# HOW: Enables object versioning

resource "aws_s3_bucket_versioning" "state_versioning" {
  bucket = aws_s3_bucket.backend_state.bucket

  versioning_configuration {
    status = "Enabled"
  }
}

# ------------------------------------------
#  ENCRYPTION FOR STATE BUCKET
# WHAT: Enables server-side encryption
# WHY: Protect sensitive state data at rest

resource "aws_s3_bucket_server_side_encryption_configuration"
"state_encryption" {
  bucket = aws_s3_bucket.backend_state.bucket

  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "AES256"
      # Advanced Encryption Standard with a 256-bit key
      # Most secure and widely used symmetric encryption algorithm
    }
  }
}

# ------------------------------------------
#  DYNAMODB TABLE FOR STATE LOCKING
# WHAT: Prevents multiple users modifying state simultaneously
# WHY: Avoids race conditions, corruption

resource "aws_dynamodb_table" "state_lock_table" {
  name         = "my-app-locks"

  # You only pay for read/write units used
  # Cost-effective and ideal for
  # low-frequency operations like Terraform runs
  billing_mode = "PAY_PER_REQUEST"

  hash_key     = "LockID"
```

```
  attribute {
    name = "LockID"
    type = "S"
  }
}

# ------------------------------------------
#  REMOTE BACKEND CONFIGURATION (IN USERS MODULE)
# WHAT: Defines how Terraform connects to S3 state
# WHY: Enables collaboration and locking
# WHEN: Must be added to the root module

terraform {
  backend "s3" {
    bucket         = "my-app-backend-state"
    key            = "my-app/backend-state"
    region         = "us-east-1"
    dynamodb_table = "my-app-locks"
    encrypt        = true
  }
}

# ------------------------------------------
#  IAM USER RESOURCE EXAMPLE
# WHAT: Creates an IAM user per environment
# WHY: Demonstrates use of terraform.workspace
# HOW: Workspace controls prefix (e.g., dev_...)

resource "aws_iam_user" "my_iam_user" {
  name = "${terraform.workspace}_my_iam_user_abc"
}

# ------------------------------------------
#  OUTPUT BLOCK
# WHAT: Shows complete IAM user details
# WHY: Useful to inspect result after apply

output "my_iam_user_complete_details" {
  value = aws_iam_user.my_iam_user
}
```

Q5 Remote Backend FAQ

What is the Advantage of Remote Backend?

- Enable Safe Team Collaboration: Multiple team members can share and access the same Terraform state file without conflicts
- Support State Locking: Prevents concurrent apply operations by locking the state file during changes
- Ensure Automatic Backups and Versioning: Maintain history of state changes for rollback, and recovery
- Secure State Storage with Encryption: Store sensitive infrastructure data securely
- Enable CI/CD and Automation: Remote backends allow pipelines and automation tools to access and update infrastructure consistently
- Reduce Risk of Accidental Deletion or Loss: No risk of losing the state file if there is a problem with a developer laptop

Why is Locking Needed?

- Prevents Concurrent Changes: Ensures that only one user or pipeline can modify the state at a time, avoiding conflicts and corruption
- Avoids Race Conditions: Stops multiple terraform apply operations from making overlapping or contradictory changes
- Essential for Team Collaboration: Helps teams work safely on shared infrastructure without stepping on each other's changes

How does Locking Work?

- Step 1: User 1 runs terraform apply. Terraform tries to acquire a lock by writing a new item with a lock into the DynamoDB table
- Step 2: If no existing lock is found, the write succeeds. User 1 successfully acquires the lock and begins modifying infrastructure
- Step 3: At the same time, User 2 also runs terraform apply. Terraform checks for a lock in the DynamoDB table
- Step 4: Terraform finds the existing lock (held by User 1). User 2 cannot proceed
- Step 5: User 1 completes the operation. Terraform automatically deletes the lock entry from the DynamoDB table
- Step 6: User 2 retries, finds the lock is now released, acquires the lock, and proceeds with their operation
- Step 7 (Optional): If a Terraform process crashes or is force-terminated, the lock remains in DynamoDB. A manual unlock using terraform force-unlock is required to proceed

Q6 Scenarios with Terraform State

- Scenario: Can we use Git for State Management?

  Git is Not a State Store — It's a Code Store: Use Git for managing your .tf code, not your .tfstate file

  (Disadvantage) Exposes Sensitive Data: State files often contain secrets, passwords, and tokens — committing them to Git risks data leaks

  (Disadvantage) No Support for Locking or Concurrency: Git can't prevent two users from pushing conflicting state updates

- Scenario: What happens if you lose your Terraform state file?

  Terraform May Forget Everything It Created: Without the state file, Terraform no longer knows what resources it is managing — it may try to recreate everything

  Can Lead to Duplicate or Conflicting Resources: Running terraform apply without the state may provision new resources instead of updating existing ones

  Orphaned Resources: Existing infrastructure becomes "orphaned" and must be managed manually or through other tools since Terraform no longer tracks it

  Best Practice: Always use remote backends with versioning and locking enabled, and back up state files regularly

- Scenario: How can you recover/update a state file?

  Recover from .tfstate.backup File: Use the automatically created backup file in the local .terraform/ directory (.tfstate.backup is an automatic backup of the previous terraform.tfstate file, created before each Terraform operation to help recover from accidental state corruption or loss)

  Restore from Remote Backend Versioning: Use previous versions stored in remote backend (if versioning is enabled)

  Rebuild State Using terraform import: Re-associate existing infrastructure with your Terraform code by importing resources manually (terraform import aws_instance.example i-0123456789abcdef0)

  Note: Time consuming, error-prone and complex

  Use terraform apply -refresh-only to Re-Sync Values: Reconnect to live infrastructure to refresh the values without making changes (only partial recovery for minor changes)

  You have an EC2 instance managed by Terraform. Someone manually changed a tag on the instance. When you execute terraform apply -refresh-only, Terraform updates the state file to reflect the current actual infrastructure (updated tag changes)

  Best Practices:

  Use remote backends with versioning and locking

  Always back up your state file regularly

Q7 How do you migrate a state file from a local backend to a remote backend?

- Configure the Remote Backend: Update your Terraform configuration to specify the remote backend
- Initialize Terraform: Run terraform init -migrate-state, Terraform will detect the backend change
- State Migration Prompt: Terraform prompts whether you want to migrate the existing local state to the remote backend
- Confirm Migration: Confirm the migration, Terraform uploads the state to the remote backend and switches to using the remote state
- Verify Migration: Ensure state is correctly migrated by running terraform plan and confirming no unexpected changes

Example backend block migrating to S3:

- Update your Terraform configuration
- Run terraform init -migrate-state

HCL / Terraform

```
# backend.tf - After preparing backend
  # Add remote backend configuration
  backend "s3" {
    bucket         = "my-terraform-state-bucket"  # Use actual bucket name
    key            = "infrastructure/terraform.tfstate"
    region         = "us-west-2"
    dynamodb_table = "terraform-state-locks"
    encrypt        = true
  }
```

HCL / Terraform

```
$ terraform init -migrate-state

Initializing the backend...
Do you want to copy existing state to the new backend?
  Pre-existing state was found while migrating the previous "local" backend to the
  newly configured "s3" backend. No existing state was found in the newly
  configured "s3" backend. Do you want to copy this state to the new "s3"
  backend? Enter "yes" to copy and "no" to start with an empty state.

  Enter a value: yes

Successfully configured the backend "s3"! Terraform will automatically
use this backend unless the backend configuration changes.
```

Q8 What is the need for Data Source of Type Terraform Remote State?

- Understanding Need to Share Outputs Across Terraform Projects: In large systems, one Terraform project may need values (like VPC IDs or bucket names) created by another — manually copying them leads to errors
- What is terraform_remote_state: A data source used to read outputs from another Terraform project's remote state file
- Enables Cross-Project Communication in Terraform: Helps connect multiple Terraform configurations without hardcoding values
- Share Outputs Like VPC IDs, Subnet IDs, or ARNs:
  - One project creates infrastructure
  - Another reads its outputs using terraform_remote_state
- Read-Only Access to External State: Pulls only the outputs, not internal details or full resource metadata
- Keeps Projects Modular and Decoupled: Avoids monolithic Terraform code and makes configurations easier to manage

HCL / Terraform

```
#  DATA SOURCE BLOCK
# WHAT: Reads remote state from S3
# - Remote state file must exist already
# - The source project must define:
#   output "vpc_id" { value = aws_vpc.main.id }
# - Helps organize large infra into modules/stages

data "terraform_remote_state" "network" {
  backend = "s3"

  config = {
    bucket = "my-terraform-states"
    key    = "network/terraform.tfstate"
    region = "us-east-1"
  }
}

output "vpc_id" {
  value = data.terraform_remote_state.network.outputs.vpc_id
  description = "VPC ID from network stack"
}
```

CHAPTER 4

Terraform Best Practices

8 Questions & Answers

4 Terraform Best Practices

Q1 What are the different file-types used in Terraform?

- .tf – Define Infrastructure Using HCL:
  Most common file type used for defining providers, resources, variables, outputs, locals, and modules
  Example: main.tf, variables.tf, outputs.tf
  All .tf files in a folder are merged automatically
- .tf.json – Machine-Readable Configuration Format:
  Same as .tf but written in JSON — useful for tools that generate Terraform code programmatically
  Rarely used manually by humans
- .tfvars – Pass External Input Values:
  Used to assign values to variables declared in .tf files
  Helps customize configurations for different environments
  Example: terraform.tfvars (default), dev.tfvars
  terraform apply -var-file=dev.tfvars
- terraform.tfstate – Track Real Infrastructure:
  Stores the current known state of managed infrastructure
  Should be stored securely (use remote backends)
- terraform.tfstate.backup:
  Backup copy of the previous state to help recover from accidental changes or failures
- .terraform.lock.hcl – Dependency Lock File:
  Records the exact versions of Terraform providers used in the current configuration
  Ensures consistent runs across machines and teams
  Automatically created and updated by Terraform during init
  Should be committed to version control
- .terraform/ – Hidden Directory:
  Stores cached provider plugins and module metadata
  Created by terraform init
  Can be deleted safely (Terraform will recreate it)
  Do NOT version control

```
HCL / Terraform
```

```
# This file is maintained automatically by "terraform init".
# Manual edits may be lost in future updates.

provider "registry.terraform.io/hashicorp/aws" {
  version     = "5.61.0"
  constraints = "SOME_VALUE"
  hashes = [
    "HASHES",
    "HASHES",
  ]
}
```

Q2 Can you recommended a Terraform Project File Structure?

- Understanding Need for Organized Terraform Projects: As infrastructure grows, unorganized code becomes hard to maintain, reuse, or debug — a clean file structure improves readability, modularity, and team collaboration
- main.tf – Core Resource Definitions: Contains the main infrastructure resources (e.g., compute, storage, networking)
- variables.tf – Input Variables: Declares all variable blocks used in the project, with optional types and defaults
- outputs.tf – Output Values: Defines values (like IDs, IPs) to be shared or referenced externally
- provider.tf – Provider Configuration: Configures providers like AWS, Azure, Google Cloud, and authentication settings
- locals.tf – Local Values and Reusable Expressions: Stores constants or calculated values
- terraform.tfvars – Input Variable Assignments: Supplies values for variables declared in variables.tf (e.g., environment-specific values)
- backend.tf – Remote Backend Configuration: Configures state storage in remote services like S3
- (Best Practice) Keep Each File Focused: Separating concerns makes maintenance easier

HCL / Terraform

```
terraform-project/
 main.tf          # Primary configuration file defining resources
 providers.tf     # Provider declarations (e.g AWS, Azure etc..)
 backend.tf       # State Backend configuration (e.g Amazon S3, ..)
 data.tf          # Data declarations
 variables.tf     # Variable declarations
 outputs.tf       # Output definitions
 locals.tf        # To define local values
 terraform.tfvars # Variable values
 README.md        # Documentation
```

Q3 What are Terraform Modules?

- Understanding Need for Reusability in Infrastructure Code: Writing the same Terraform code for VMs, VPCs, or databases across multiple projects leads to duplication and inconsistency
- What Are Terraform Modules: A collection of .tf files grouped together to perform a specific task — like provisioning a server or setting up a network — which can be reused across different Terraform configurations
- Helps Simplify, Organize, and Reuse Infrastructure Code: Modules allow you to break down large configurations and promote consistency across environments
- Reuse Common Infrastructure Patterns:
  - Create a reusable module for VPC setup or EC2 instances
  - Use the same module in different applications (with different inputs)
- Separate Logic for Better Maintainability:
  - Break complex configs into smaller, focused modules (e.g., network, database, compute)
- Share Infrastructure Configuration Across Teams:
  - Teams can publish modules to Terraform Registry or internal repositories for reuse
- Accepts Inputs and Produces Outputs: Use variable and output blocks inside the module to customize behavior and return values
- Supports Local and Remote Sources: Modules can be stored locally (./modules/vpc) or remotely (GitHub, Terraform Registry)

DIFFERENT TYPES OF MODULES

- Root Module: The module defined in the root of the Terraform configuration directory where you run Terraform commands
- Child Modules (Local Modules): Modules stored locally in your project directory, reusable across configurations
- Registry Modules: Published and versioned modules hosted on public or private registries, like the official Terraform Registry
- Remote Modules: Modules fetched from remote sources like GitHub, ..

Example:

```
HCL / Terraform
```

```
# Let's create a User Module
# Purpose: To create IAM users for your application
# Input: Name of the application
# Output: Details of the created IAM user (e.g., name, ARN)

# PROJECT STRUTURE
# terraform-project/
#  app-a/
#     main.tf         # Uses the IAM user module for app-a
#
#  app-b/
#     main.tf         # Uses the IAM user module for app-b
#
#  modules/
#     users/
#         main.tf     # Defines IAM user resource
#         outputs.tf  # Outputs full IAM user details
#         README.md   # Module documentation (optional)
#
#  README.md                    # Project-level documentation (optional)


#  BENEFITS OF THIS STRUCTURE
# - Reusable IAM user logic in users module
# - Easy to re-use across apps or projects
# - Clean separation between logic and usage


# ---------------------------------------------
#  modules/users/main.tf

variable "application" {
  default = "default"
}

provider "aws" {
  region = "us-east-1"
}

# This can be a complex resource
# Network or EC2 Instance or ..
resource "aws_iam_user" "my_iam_user" {
  name = ""iam_user_for_${var.application}_application"
}

# ---------------------------------------------
#  OUTPUT BLOCK
#  modules/users/outputs.tf

output "my_iam_user_complete_details" {
  value = aws_iam_user.my_iam_user
}

# ---------------------------------------------
#  MODULE USAGE - APP A
#  app-a/main.tf
# WHAT: Uses the IAM user module
# WHY: Creates user specific to "app-a"

module "user_module" {
  source      = "../modules/users"
  application = "app-a"
}

# OTHER app-a RESOURCES GO HERE

# ---------------------------------------------
#  MODULE USAGE - APP B
#  app-b/main.tf
# WHAT: Uses the same module
# WHY: Creates user specific to "app-b"
```

```
module "user_module" {
  source      = "../modules/users"
  application = "app-b"
}

# OTHER app-b RESOURCES GO HERE
```

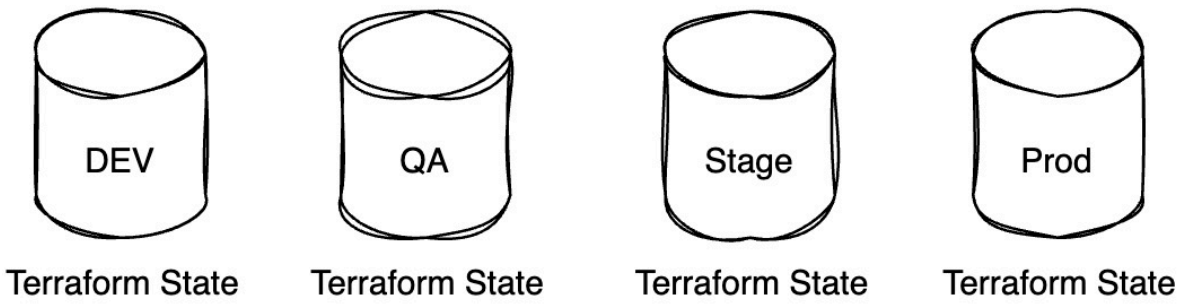
HCL / Terraform

```
# PROJECT STRUTURE
# terraform-project/
#  main.tf
#  variables.tf
#  provider.tf
#  others.tf
#  modules/
#     vpc/
#         main.tf
#         outputs.tf
#         README.md
#     ec2/
#         main.tf
#         outputs.tf
#         README.md
#     rds/
#         main.tf
#         outputs.tf
#         README.md
#
#  README.md
```

HCL / Terraform

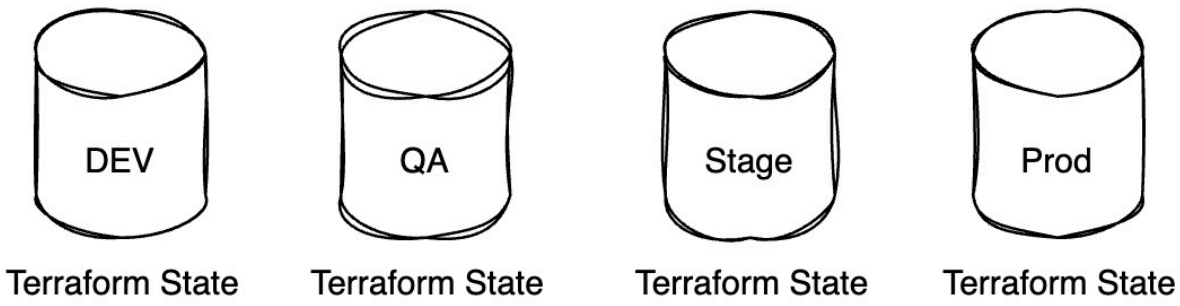
```
module "vpc" {
  source = "./modules/vpc"                # Local module
}

module "network" {
  source = "terraform-aws-modules/vpc/aws" # Registry module
  version = "3.11.0"
}

module "custom" {
  source = "git::https://github.com/org/repo.git//modules/custom"
}
```

Q4 What is the need for Terraform Workspaces?



- Understanding Need to Manage State in Multiple Environments: Dev, QA, and Prod environments often share the same Terraform code but require different state files
- Terraform Workspace: A named instance of Terraform's state file that allows you to use the same configuration to manage multiple environments with isolated state
- Keeps Separate State Files for Each Workspace: Stored under .terraform/ or managed by the remote backend
- Manage Dev, QA, Stage and Prod with Same Code: Use different workspaces to provision similar infrastructure across environments
- Test Infrastructure Changes Safely: Try new changes in a test workspace without affecting production
- Create a New Workspace: terraform workspace new dev – Creates and switches to a new workspace named dev
- List All Workspaces: terraform workspace list – Shows all existing workspaces; current one is marked with *
- Select an Existing Workspace: terraform workspace select prod – Switches to the prod workspace
- Delete a Workspace: terraform workspace delete test – Deletes the test workspace (you must switch out of it first)
- Show Current Workspace: terraform workspace show – Displays the name of the currently active workspace
- Use Workspaces for Lightweight Multi-Env Setups: Perfect for small-to-medium projects that want to avoid repetition but still need environment isolation

Q5 Explain a Sample Step By Step Workflow of Using Two Workspaces



Terraform State For Difference Environments

- Create Two Workspaces – Dev and QA
  - terraform workspace new dev – Creates and switches to dev workspace
  - terraform workspace new qa – Creates and switches to qa workspace
- Switch to Dev Workspace
  - terraform workspace select dev – Activate the dev workspace
  - Run terraform plan and terraform apply to provision resources for Dev
- Switch to QA Workspace
  - terraform workspace select qa – Switch to the qa workspace
  - Run terraform plan and terraform apply to create separate QA infrastructure
- Verify Workspace-Specific State
  - Each workspace has its own isolated terraform.tfstate
  - Resources in dev and qa are managed independently even though they share the same .tf configuration
- Destroy QA Resources Without Affecting Dev
  - terraform workspace select qa
  - terraform destroy – Only deletes QA infrastructure
  - Dev resources remain untouched in their own workspace
- (Best Practice) Use Workspaces + Variable Files
  - Example: terraform apply -var-file=dev.tfvars in dev, and qa.tfvars in qa
  - Ensures proper configuration per environment while keeping code consistent

Q6 When do you use Terraform Enterprise?

- What is Terraform Enterprise: A self-hosted or SaaS offering from HashiCorp that provides additional features for teams using Terraform at scale
- Manage Large Teams and Projects at Scale: Offers scalable architecture with workspaces, organizations, and user management
- Role-Based Access Control (RBAC): Manage who can view, plan, apply, or modify infrastructure across workspaces and teams
- Policy Enforcement: Enforce compliance rules (e.g., no public S3 buckets, region restrictions) before changes are applied
- Private Module Registry: Share internal modules securely across teams
- Remote Operations with Secure State Management: Avoid local state files with encrypted, versioned remote backends
- If You Need an Internal, Secure Deployment: Use Self-hosted Terraform Enterprise

Q7 Explain a Few Terraform Best Practices

- Automate with CI/CD: Integrate Terraform with CI/CD pipelines to automate validation, planning, and application of infrastructure changes
- Use Version Pinning for Providers and Terraform: Lock specific versions to avoid unexpected breaking changes
- Organize Code into Modules: Break infrastructure into reusable, well-scoped modules for better maintainability
- Use Remote Backends with Locking: Store state files in a remote backend with locking and encryption
- Use terraform workspace for Multiple Environments:Isolate dev/staging/prod environments using named workspaces or separate state files and directories
- Implement Resource Tagging: Apply consistent tags (e.g., Environment, Owner, Application) to all resources for better tracking, cost visibility, and compliance
- Avoid Hardcoding – Use Variables and tfvars Files: Keep your code flexible and environment-agnostic by using input variables and separate value files for each environment
- Never Commit State Files or Secrets to Git: Add terraform.tfstate and .terraform/ to .gitignore to protect your credentials
- Limit Use of provisioners: Avoid provisioner blocks when possible — they're considered a last resort. Prefer configuration management tools.

Q8 What are Immutable Servers?

1. The Problem with Mutable Servers (Traditional Approach): Traditionally, servers were provisioned once and then "tweaked" over time. If you needed to make a change (e.g., install new software, apply a security patch, update a configuration file):

- You would log into the existing server
- Execute a script or manual commands to apply the changes

This approach creates several problems:

- Configuration Drift: The actual state of the server deviates from its original, documented configuration. Different servers, even those intended to be identical, can gradually accumulate unique configurations and issues.
- Reproducibility Challenges: If you need to create a new server with the exact same updated configuration, you might have to manually re-run a sequence of undocumented or lost scripts, which is prone to errors.
- Debugging Difficulty: It becomes hard to trace exactly what changes were applied over time, complicating debugging.

2. The Solution: Immutable Servers with Infrastructure as Code: Immutable servers are servers that, once provisioned, are never modified. If any change is needed, a new server is provisioned with the updated configuration, and the old server is decommissioned.

- Process:

  Your Infrastructure as Code (e.g., Terraform) configuration is updated with the desired changes (e.g., a new software version, a security patch)

  A new server (or set of servers) is provisioned from this updated configuration

  Once the new server(s) are fully operational and verified, traffic is redirected to them

  The old server(s) are then decommissioned and destroyed

3. Advantages of Immutable Servers:

- Consistency and Reproducibility: Since servers are always created from the latest version-controlled configuration, there is no configuration drift. Every server is identical to others provisioned from the same code, ensuring consistency across environments (development, staging, production).
- Reliability: Building new servers from scratch ensures that all dependencies and configurations are correctly applied every time. This reduces the risk of hidden issues from incremental changes.
- Simplified Rollbacks: If a new server configuration has issues, you can quickly roll back by simply redirecting traffic to the previous version's servers (if still available) or destroying the new servers and provisioning from a previous, known-good configuration.
- Easier Debugging: If a problem occurs, you know exactly what configuration was used to build the server, simplifying debugging.
- Reduced Operational Complexity: Operations teams manage a consistent, repeatable build process rather than tracking individual server states.

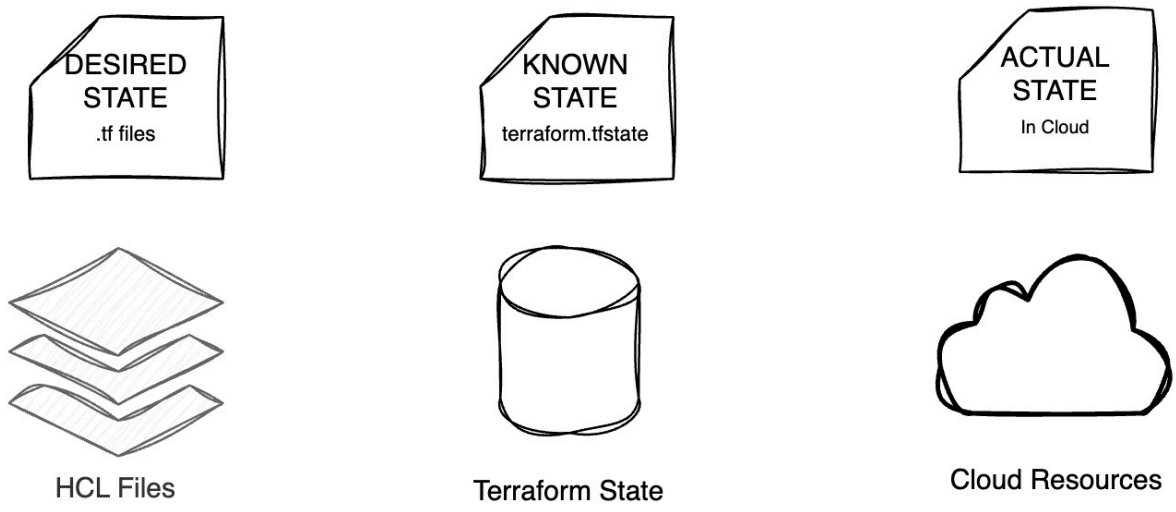
CHAPTER 5

Terraform Interview - Diagrams

4 Questions & Answers

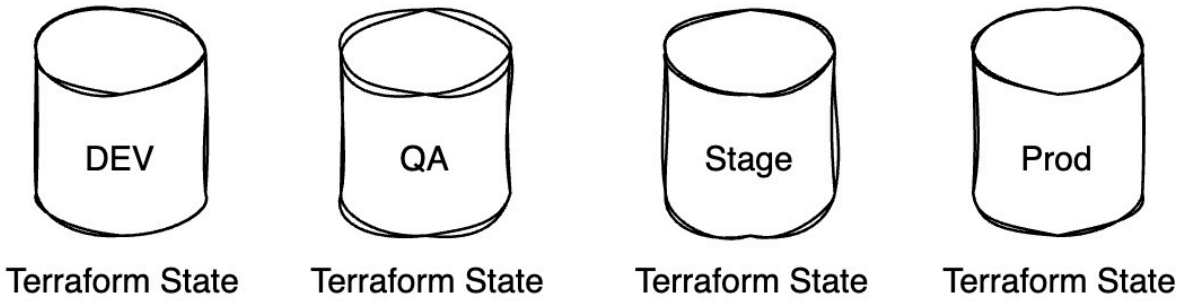
5 Terraform Interview - Diagrams

Q1 Terraform State



Terraform State

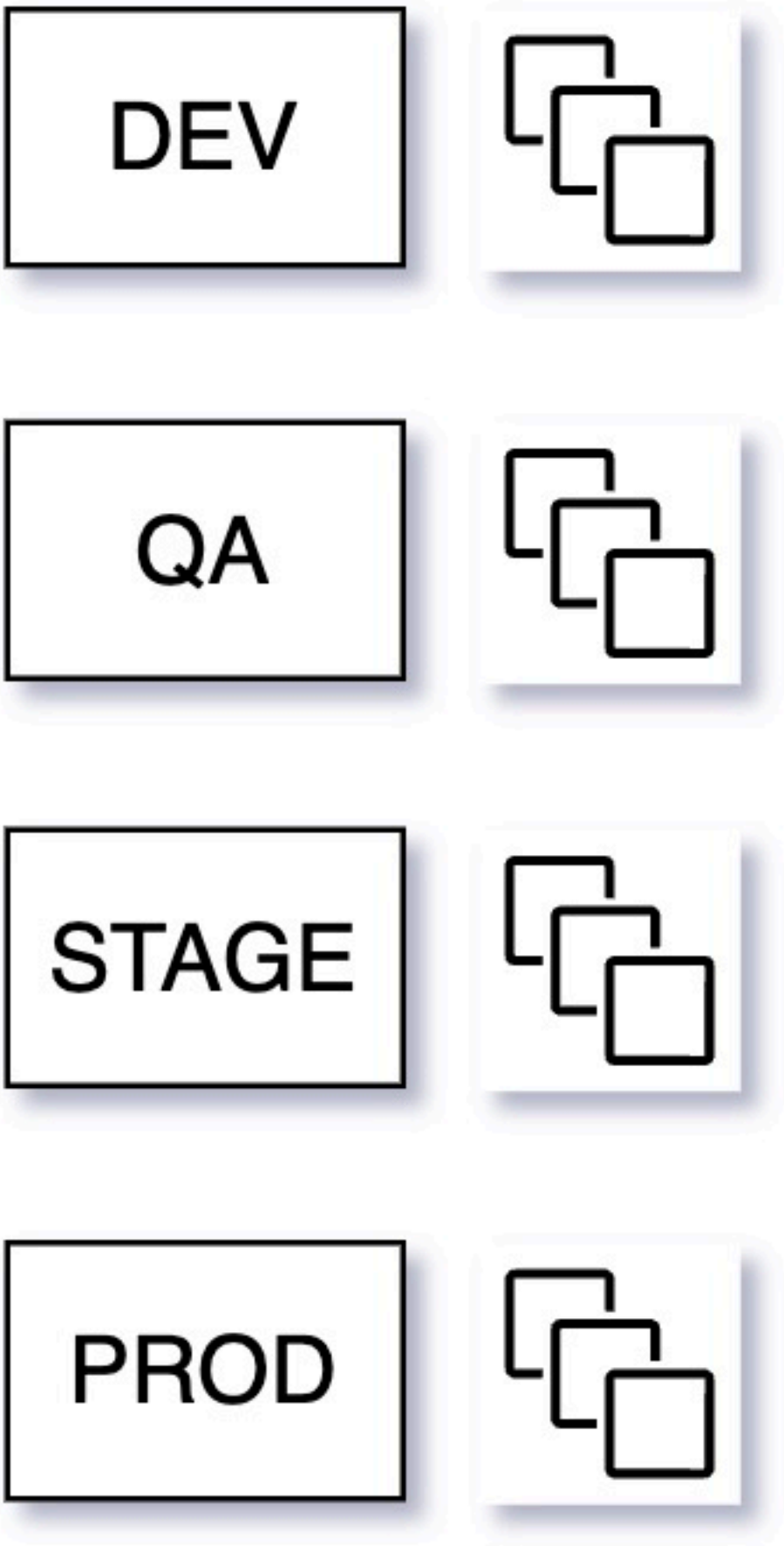
Q2 Terraform State For Difference Environments



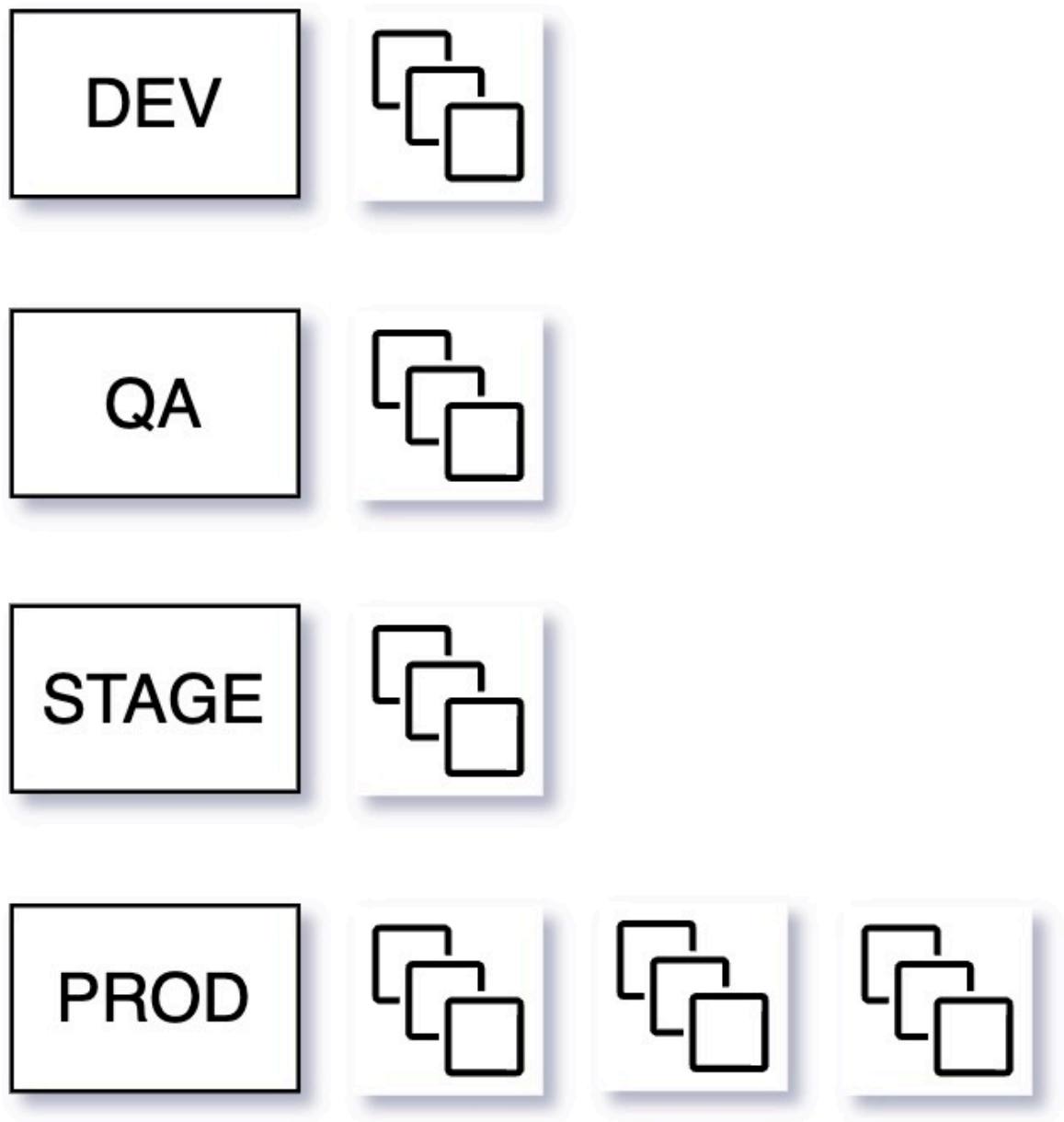
Terraform State For Difference Environments

Q3 Environments

Environments


DEV
QA
STAGE
PROD

Q4 Environments Are Not All The Same!



Environments Are Not All The Same